Cycle de formation – Machine Learning

Arbres de décision, forêts aléatoires et KNN

Comprendre les mécanismes, construire les modèles et calculer les prédictions

Savadogo Modeste
Formateur

De l'intuition à l'algorithme, puis de l'algorithme au modèle.

Objectifs pédagogiques

À la fin de la séance

1. Expliquer comment CART choisit une variable et un seuil de séparation.
2. Construire un petit arbre à la main à partir d'un jeu de données.
3. Expliquer comment une forêt combine plusieurs arbres appris sur des échantillons différents.
4. Effectuer une prédiction KNN en calculant et en classant des distances.
5. Identifier les hyperparamètres qui contrôlent le comportement de chaque méthode.
6. Comparer les modèles avec un protocole expérimental commun.

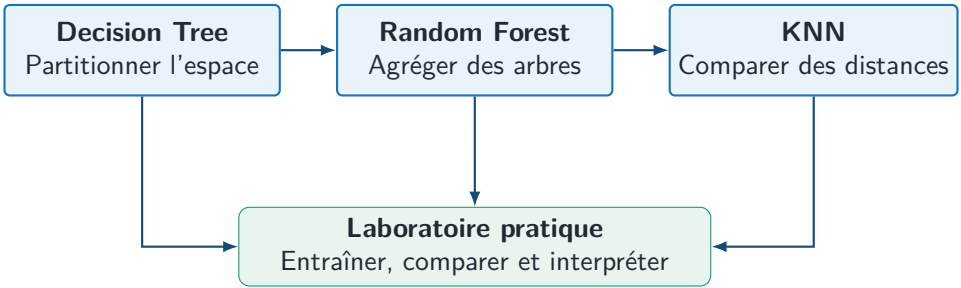
Fil conducteur

Chaque algorithme sera étudié selon la même démarche :

1. Données disponibles
2. Règle de fonctionnement
3. Calcul d'une étape
4. Construction du modèle
5. Prédiction sur une nouvelle instance

Organisation de la séance

Une progression en trois algorithmes



Positionnement : trois règles de prédiction différentes

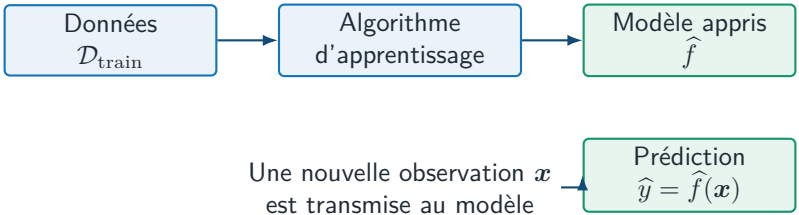
On dispose d'un ensemble d'observations étiquetées :

$$\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{m}, \qquad \boldsymbol{x}_i \in \mathbb{R}^p.$$

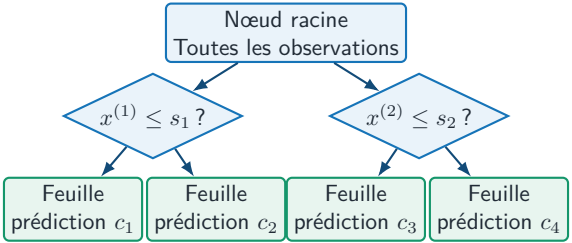
Arbre de décision	Forêt aléatoire	KNN
Une suite de tests sur les caractéristiques conduit à une feuille.	Plusieurs arbres produisent des prédictions ensuite agrégées.	Les étiquettes des observations les plus proches déterminent la prédiction.

Deux tâches possibles

Classification : y prend une valeur dans un ensemble de classes. **Régression :** $y \in \mathbb{R}$.



- Les **paramètres** sont déterminés pendant l'apprentissage (par exemple les seuils et les feuilles d'un arbre).
- Les **hyperparamètres** sont fixés avant ou pendant la sélection du modèle (profondeur maximale, nombre d'arbres, nombre de voisins).



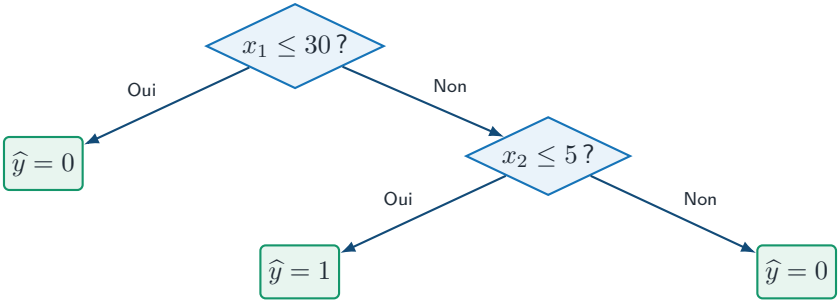
Définition

Un arbre de décision est un modèle dont chaque nœud interne teste une condition sur une caractéristique. Le résultat du test détermine le fils à suivre. Chaque feuille fournit la prédiction.

Arbre de décision : comment obtenir une prédiction ?

Une observation suit un chemin unique

Considérons un arbre dont les tests sont :



Pour $\boldsymbol{x} = (42, 3)$:

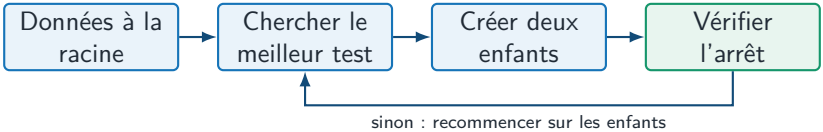
1. $42 \leq 30$? Non : aller à droite.
2. $3 \leq 5$? Oui : atteindre la feuille de gauche.

Prédiction

La prédiction est la valeur portée par la feuille atteinte : $\widehat{y} = 1$.

L'algorithme construit l'arbre à partir du dataset d'entraînement.

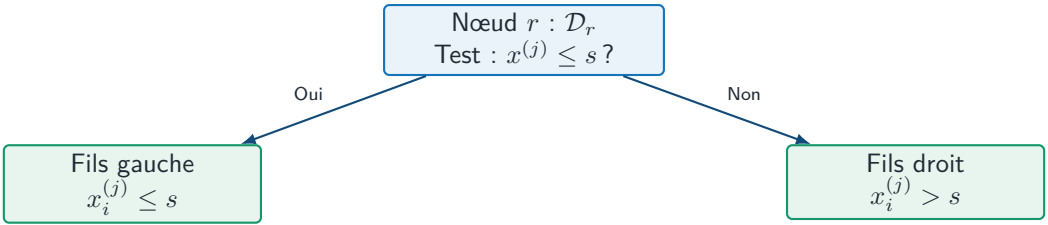
1. **Initialisation :** créer la racine et lui associer toutes les observations.
2. **Choix du meilleur test :** sélectionner une caractéristique et un seuil.
3. **Division du nœud :** répartir les observations dans deux sous-ensembles.
4. **Partitionnement récursif :** répéter la procédure sur les nœuds enfants.
5. **Arrêt :** terminer lorsque le critère d'arrêt est satisfait.



Un nœud est séparé par un test binaire

Variable séparatrice et seuil

À un nœud r, les observations forment un ensemble $\mathcal{D}_r$.



$$\mathcal{D}_L = \{(\boldsymbol{x}_i, y_i) \in \mathcal{D}_r : \ x_i^{(j)} \leq s\} \qquad \mathcal{D}_R = \{(\boldsymbol{x}_i, y_i) \in \mathcal{D}_r : \ x_i^{(j)} > s\}$$

- $x^{(j)}$: caractéristique choisie pour effectuer la séparation.
- s : seuil auquel la valeur de cette caractéristique est comparée.

Supposons qu'une caractéristique prenne les valeurs distinctes :

$$v_1 < v_2 < \cdots < v_q.$$

Les seuils candidats sont les milieux entre deux valeurs distinctes consécutives :

$$s_k = \frac{v_k + v_{k+1}}{2}, \qquad k = 1, \ldots, q-1.$$

Exemple

Valeurs observées : $\{1, 1, 2, 2, 3, 5\}$. Valeurs distinctes : $1 < 2 < 3 < 5$.

$$\mathcal{S} = \left\{ \frac{1+2}{2}, \frac{2+3}{2}, \frac{3+5}{2} \right\} = \{1{,}5;\ 2{,}5;\ 4\}.$$

Conséquence

Il suffit de tester les seuils situés entre les valeurs distinctes : les observations identiques ne peuvent pas être séparées par un seuil.

Dans une feuille de régression, la prédiction constante qui minimise la somme des carrés est la moyenne des valeurs cibles.

$$\overline{y}_L = \frac{1}{|\mathcal{D}_L|} \sum_{i \in L} y_i, \qquad \overline{y}_R = \frac{1}{|\mathcal{D}_R|} \sum_{i \in R} y_i.$$

Pour un test candidat (j, s) :

$$\boxed{E(j,s) = \sum_{i \in L} (y_i - \overline{y}_L)^2 + \sum_{i \in R} (y_i - \overline{y}_R)^2}$$

- Calculer la moyenne cible dans chaque fils.
- Calculer la somme des erreurs quadratiques dans chaque fils.
- Additionner les deux erreurs : le test retenu minimise $E(j, s)$.

Exemple CART : six observations, une caractéristique

Données de l'exercice – profondeur maximale $d_{\max} = 2$

Instance	x_i	y_i
1	2	1
2	1	2
3	0,5	6
4	2	3
5	3	4
6	1	2

Problème

Construire un arbre CART de régression à partir de ces observations. La profondeur maximale autorisée est 2.

- Une seule caractéristique : x.
- Cible quantitative : y.
- La prédiction d'une feuille est la moyenne des y_i qui y arrivent.

Première question

Quelle est l'erreur quadratique à la racine, avant toute séparation ?

Étape 1 – Erreur quadratique à la racine

La feuille initiale prédit la moyenne globale

La moyenne des six étiquettes est :

$$\overline{y} = \frac{1+2+6+3+4+2}{6} = 3.$$

L'erreur quadratique à la racine vaut :

$$\begin{aligned} E(\mathcal{D}) &= \sum_{i=1}^{6} (y_i - 3)^2 \\ &= (1-3)^2 + (2-3)^2 + (6-3)^2 \\ &\quad + (3-3)^2 + (4-3)^2 + (2-3)^2 \\ &= 4 + 1 + 9 + 0 + 1 + 1. \end{aligned}$$

Résultat à la racine

$$\boxed{E(\mathcal{D}) = 16}$$

Étape 2 – Seuils candidats à la racine

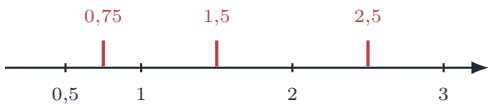
Trier les valeurs de la caractéristique

Les valeurs distinctes triées sont :

$$0{,}5 < 1 < 2 < 3.$$

Les seuils candidats sont donc :

$$\mathcal{S} = \left\{ \frac{0{,}5+1}{2}, \frac{1+2}{2}, \frac{2+3}{2} \right\} = \boxed{\{0{,}75;\ 1{,}5;\ 2{,}5\}}.$$



Chaque seuil définit une séparation $x \leq s$ / $x > s$. Il faut calculer l'erreur des deux fils pour chacun.

Étape 3 – Tester le seuil $s = 0{,}75$

Une observation à gauche, cinq à droite

Fils gauche : $x \leq 0{,}75$

Une seule observation : $(0{,}5; 6)$.

$$\overline{y}_L = 6, \qquad E_L = 0.$$

Fils droit : $x > 0{,}75$

Étiquettes : $\{1, 2, 3, 4, 2\}$.

$$\overline{y}_R = \frac{12}{5} = 2{,}4.$$

$$\begin{aligned} E_R &= (1 - 2{,}4)^2 + (2 - 2{,}4)^2 \\ &\quad + (3 - 2{,}4)^2 + (4 - 2{,}4)^2 \\ &\quad + (2 - 2{,}4)^2 \\ &= 1{,}96 + 0{,}16 + 0{,}36 \\ &\quad + 2{,}56 + 0{,}16 \\ &= 5{,}2. \end{aligned}$$

Erreur du partitionnement

$$\boxed{E(0{,}75) = 0 + 5{,}2 = 5{,}2}$$

Étape 3 – Tester le seuil $s = 1{,}5$

Trois observations à gauche, trois à droite

Fils gauche : $x \leq 1{,}5$

Étiquettes : $\{6, 2, 2\}$.

$$\overline{y}_L = \frac{10}{3}, \qquad E_L = \frac{96}{9} = \frac{32}{3}.$$

Fils droit : $x > 1{,}5$

Étiquettes : $\{1, 3, 4\}$.

$$\overline{y}_R = \frac{8}{3}, \qquad E_R = \frac{42}{9} = \frac{14}{3}.$$

$$\begin{aligned} E(1{,}5) &= \frac{32}{3} + \frac{14}{3} \\ &= \frac{46}{3} \\ &\approx 15{,}33. \end{aligned}$$

Interprétation

Cette séparation réduit très peu l'erreur par rapport à l'erreur initiale de 16.

Étape 3 – Tester le seuil $s = 2{,}5$

Cinq observations à gauche, une à droite

Fils gauche : $x \leq 2{,}5$

Étiquettes : $\{6, 2, 2, 1, 3\}$.

$$\overline{y}_L = \frac{14}{5} = 2{,}8.$$

$$E_L = 10{,}24 + 0{,}64 + 0{,}64 + 3{,}24 + 0{,}04 = 14{,}8.$$

Fils droit : $x > 2{,}5$

Étiquette : $\{4\}$.
$\overline{y}_R = 4$ et $E_R = 0$.

Erreur du partitionnement

$$\boxed{E(2{,}5) = 14{,}8 + 0 = 14{,}8}$$

- $E(0{,}75) = 5{,}2$
- $E(1{,}5) \approx 15{,}33$
- $E(2{,}5) = 14{,}8$

Étape 4 – Choisir le test de la racine

Le seuil $0{,}75$ minimise l'erreur après séparation

Seuil s	E_L	E_R	$E(s)$
0,75	0	5,20	5,20
1,5	10,67	4,67	15,33
2,5	14,80	0	14,80

Meilleur test à la racine

$$\boxed{x \leq 0{,}75} \quad \text{car} \quad \arg\min_{s \in \mathcal{S}} E(s) = 0{,}75.$$

La réduction de l'erreur quadratique est :

$$\Delta E = E(\mathcal{D}) - E(0{,}75) = 16 - 5{,}2 = 10{,}8.$$

- Fils gauche : une observation, $y = 6$; ce nœud est déjà homogène.
- Fils droit : cinq observations ; on poursuit la construction.

Étape 5 – Développer le fils droit

Rechercher le meilleur seuil dans le sous-ensemble restant

Après la séparation à $0{,}75$, le fils droit contient :

$$\{(1,2),(1,2),(2,1),(2,3),(3,4)\}.$$

Valeurs distinctes de x : $1 < 2 < 3$. Seuils candidats : $1{,}5$ et $2{,}5$.

Test $x \leq 1{,}5$

Gauche : $\{2,2\}$, SSE $= 0$.
Droite : $\{1,3,4\}$, moyenne $8/3$.
SSE $= \frac{14}{3} \approx 4{,}67$.

Test $x \leq 2{,}5$

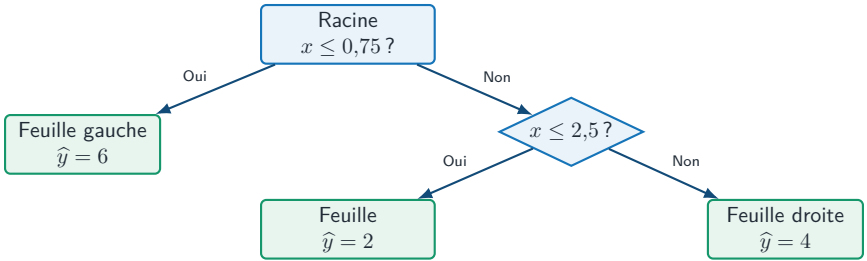
Gauche : $\{2,2,1,3\}$, moyenne 2.
SSE $= 0 + 0 + 1 + 1 = 2$.
Droite : $\{4\}$, SSE $= 0$.

Choix au fils droit

$$\boxed{s^* = 2{,}5} \quad \text{car} \quad 2 < \frac{14}{3}.$$

Arbre final obtenu avec $d_{\max} = 2$

La construction récursive est terminée



Fonction de prédiction explicite

$$\widehat{f}(x) = \begin{cases} 6, & x \leq 0{,}75, \\ 2, & 0{,}75 < x \leq 2{,}5, \\ 4, & x > 2{,}5. \end{cases}$$

CART en classification : qu'est-ce qui change ?

La logique de partitionnement reste identique

Pour la classification, on ne minimise plus une somme d'erreurs quadratiques. On mesure l'impureté des classes dans chaque nœud.

Impureté de Gini

$$G(D) = 1 - \sum_{c=1}^{C} p_c^2$$

p_c est la proportion de la classe c dans le nœud.

Entropie

$$H(D) = -\sum_{c=1}^{C} p_c \log_2(p_c)$$

Convention : $0 \log_2(0) = 0$.

Pour un test qui produit deux fils :

$$I_{\text{après}} = \frac{n_L}{n} I(D_L) + \frac{n_R}{n} I(D_R).$$

Règle de sélection

Choisir le test qui minimise l'impureté pondérée des fils, ou, de manière équivalente, qui maximise la diminution d'impureté.

Exemple court : calcul de Gini

Une séparation rend les groupes plus homogènes

Un nœud contient 10 observations : 6 de classe A et 4 de classe B.

$$G(D) = 1 - \left(\frac{6}{10}\right)^2 - \left(\frac{4}{10}\right)^2 = 0{,}48.$$

Un test produit :

- fils gauche : 5 A, 0 B, donc $G(D_L) = 0$;
- fils droit : 1 A, 4 B, donc $G(D_R) = 1 - (1/5)^2 - (4/5)^2 = 0{,}32$.

$$G_{\text{après}} = \frac{5}{10}(0) + \frac{5}{10}(0{,}32) = 0{,}16.$$

Gain d'impureté

$$\boxed{\Delta G = 0{,}48 - 0{,}16 = 0{,}32}$$

Le test sépare mieux les classes que le nœud initial.

Un arbre peut continuer à créer des feuilles très spécifiques. Il peut alors mémoriser les données d'entraînement plutôt que capturer des régularités généralisables.

Arbre peu profond

- Peu de règles
- Modèle plus simple
- Risque de sous-apprentissage

Arbre très profond

- Nombreuses règles
- Ajustement très fin au train
- Risque de surapprentissage

Hyperparamètres importants

`max_depth` : profondeur maximale ; `min_samples_split` : nombre minimal d'observations pour tenter une division ; `min_samples_leaf` : nombre minimal d'observations dans une feuille ; `ccp_alpha` : force de l'élagage par complexité-coût.

Bilan : arbres de décision

Ce qu'il faut retenir

Forces

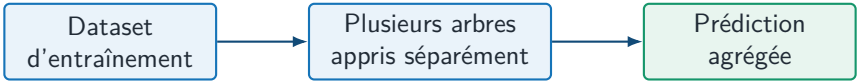
- Règles de décision lisibles.
- Relations non linéaires et interactions capturées par les divisions.
- Peu sensibles à la mise à l'échelle des variables.
- Utilisables en régression et en classification.

Limites

- Instabilité : de petites variations des données peuvent modifier l'arbre.
- Un arbre profond peut surapprendre.
- Les frontières sont construites par divisions successives, souvent parallèles aux axes.

Question de transition

Si un arbre peut être instable, peut-on réduire cette instabilité en combinant plusieurs arbres ?

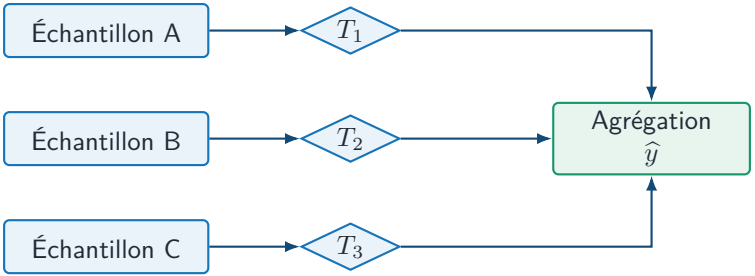


Principe

Une forêt aléatoire est un ensemble d'arbres de décision entraînés avec des mécanismes de randomisation, puis combinés pour produire une prédiction.

Objectif : réduire la dépendance à un arbre particulier et améliorer la stabilité des prédictions.

Un arbre profond peut être très sensible à l'échantillon utilisé pour l'entraînement.

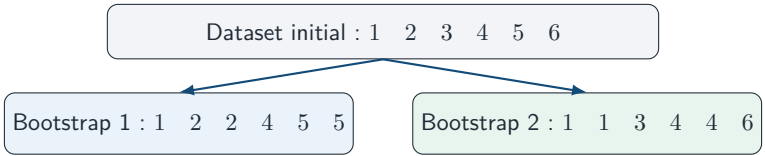


- Des arbres différents peuvent commettre des erreurs différentes.
- Une agrégation peut atténuer les fluctuations propres à un arbre.
- Le bénéfice dépend de la diversité des arbres : des arbres identiques n'apportent presque aucune réduction de variance.

Première randomisation : le bootstrap

Tirer des observations avec remise

À partir de m observations, on construit un échantillon bootstrap en effectuant m tirages avec remise.



- Une observation peut apparaître plusieurs fois.
- D'autres observations peuvent ne pas apparaître dans l'échantillon.
- Chaque échantillon bootstrap conserve la même taille que le dataset initial.

Pour une observation donnée, la probabilité de ne jamais être sélectionnée après m tirages parmi m observations est :

$$P(\text{non sélectionnée}) = \left(1 - \frac{1}{m}\right)^m.$$

Lorsque m est grand :

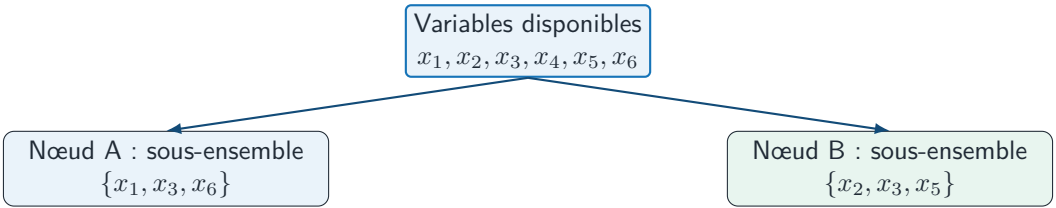
$$\left(1 - \frac{1}{m}\right)^m \longrightarrow e^{-1} \approx 0{,}368.$$

Conséquence asymptotique

Un échantillon bootstrap contient environ $63{,}2\%$ d'observations distinctes du dataset initial ; environ $36{,}8\%$ sont absentes et forment l'ensemble OOB de cet arbre.

Les observations OOB peuvent servir à estimer une erreur interne de la forêt. Cette estimation ne remplace pas un jeu de test indépendant lorsque celui-ci est nécessaire.

En plus du bootstrap, une forêt aléatoire limite les variables candidates à chaque nœud.



- Les variables considérées changent d'un nœud à l'autre.
- Cela évite que les mêmes caractéristiques dominantes imposent les mêmes divisions dans tous les arbres.
- Le mécanisme tend à diminuer la corrélation entre les arbres.

Classification : vote majoritaire

Si les arbres prédisent $\widehat{y}_b$:

$$\widehat{y} = \mathrm{mode}\{\widehat{y}_1, \ldots, \widehat{y}_B\}.$$

On peut aussi moyenner les probabilités estimées par les arbres puis appliquer un seuil ou choisir la classe la plus probable.

Régression : moyenne

$$\widehat{y} = \frac{1}{B} \sum_{b=1}^{B} \widehat{y}_b.$$

Chaque arbre donne une valeur numérique ; la forêt calcule leur moyenne.

Exemple numérique : agrégation de cinq arbres

Classification par vote majoritaire

Une nouvelle observation est transmise à cinq arbres. Les prédictions obtenues sont :

$$\underbrace{\text{Oui},\ \text{Non},\ \text{Oui},\ \text{Oui},\ \text{Non}}_{5\ \text{arbres}}.$$

Arbre	T_1	T_2	T_3	T_4	T_5	Vote
Prédiction	Oui	Non	Oui	Oui	Non	

Résultat

La classe Oui reçoit 3 votes sur 5 : la forêt prédit donc **Oui**.

En régression : si trois arbres prédisent 120, 130 et 110, la forêt prédit :

$$\widehat{y} = \frac{120 + 130 + 110}{3} = 120.$$

Pourquoi l'agrégation peut-elle réduire la variance ?

Une intuition quantitative

Supposons que chaque arbre ait une erreur de variance σ^2 et que la corrélation entre deux arbres soit approximativement ρ. Pour la moyenne de B arbres, on obtient :

$$\boxed{\mathrm{Var}(\overline{T}) = \sigma^2 \left(\rho + \frac{1-\rho}{B} \right)}.$$

- Si les arbres sont très corrélés ($\rho \approx 1$), ajouter des arbres apporte peu.
- Si les arbres sont moins corrélés, la moyenne peut réduire davantage la variance.
- Lorsque B augmente, la variance tend vers $\rho\sigma^2$ dans cette approximation.

Point essentiel

L'augmentation du nombre d'arbres ne supprime pas la composante liée à leur corrélation. Le bootstrap et la sélection aléatoire des variables servent notamment à créer de la diversité.

Cette formule suppose des variances individuelles comparables et une corrélation paire à paire approximativement constante.

Algorithme d'une Random Forest

Résumé opérationnel

1. Fixer le nombre d'arbres B.
2. Pour chaque arbre $b = 1, \ldots, B$:
   - 2.1 tirer un échantillon bootstrap du dataset d'entraînement ;
   - 2.2 construire un arbre ; à chaque nœud, considérer un sous-ensemble aléatoire de variables ;
   - 2.3 choisir la meilleure séparation parmi les variables candidates ;
   - 2.4 poursuivre jusqu'au critère d'arrêt.
3. Agréger les prédictions des B arbres.

Hyperparamètres principaux

`n_estimators` : nombre d'arbres ; `max_features` : nombre ou proportion de variables candidates par nœud ; `max_depth` : profondeur maximale ; `min_samples_leaf` : taille minimale d'une feuille ; `bootstrap` : activation du rééchantillonnage avec remise.

Bilan : forêts aléatoires

Forces et limites

Forces

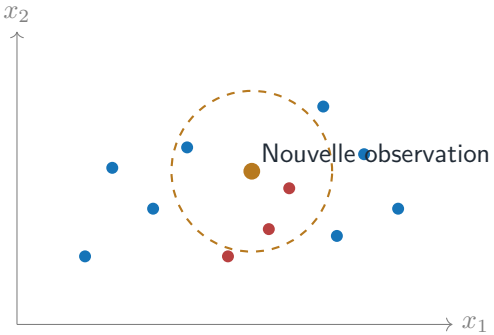
- Souvent plus stables qu'un arbre individuel.
- Capturent des relations non linéaires.
- Peu sensibles à la mise à l'échelle.
- Permettent une estimation OOB et des mesures d'importance des variables.

Limites

- Modèle moins interprétable qu'un arbre unique.
- Coût mémoire et calcul plus élevés.
- Une importance par impureté peut être biaisée par certaines variables (notamment à forte cardinalité).
- Une forêt très grande n'est pas automatiquement meilleure.

Idée fondamentale

KNN mémorise les observations d'entraînement. Pour une nouvelle observation, il recherche les k voisins les plus proches et utilise leurs étiquettes pour produire la prédiction.



KNN : une phase d'apprentissage particulière

Lazy learning

Phase d'apprentissage

1. Conserver les observations d'entraînement et leurs étiquettes.
2. Définir une mesure de distance.
3. Choisir le nombre de voisins k.

Il n'y a pas de recherche de coefficients par optimisation comme en régression linéaire.

Phase de prédiction

1. Calculer les distances entre la nouvelle observation et les observations conservées.
2. Trier les distances.
3. Sélectionner les k plus petites.
4. Agréger leurs étiquettes.

Conséquence

L'apprentissage est léger, mais la prédiction peut être coûteuse car elle nécessite de rechercher les voisins dans les données mémorisées.

KNN : règle de prédiction

Deux variantes selon la nature de la cible

Soit $N_k(\boldsymbol{x})$ l'ensemble des indices des k plus proches voisins de $\boldsymbol{x}$.

Régression : moyenne

$$\widehat{y}(\boldsymbol{x}) = \frac{1}{k} \sum_{i \in N_k(\boldsymbol{x})} y_i.$$

La prédiction est la moyenne des valeurs cibles des voisins.

Classification : vote majoritaire

$$\widehat{y}(\boldsymbol{x}) = \arg\max_{c \in \mathcal{C}} \sum_{i \in N_k(\boldsymbol{x})} \mathbb{1}(y_i = c).$$

La prédiction est la classe la plus fréquente parmi les voisins.

Pour $\boldsymbol{x}, \boldsymbol{z} \in \mathbb{R}^p$ et $q \geq 1$:

$$d_q(\boldsymbol{x}, \boldsymbol{z}) = \left(\sum_{j=1}^{p} |x^{(j)} - z^{(j)}|^q \right)^{1/q}.$$

Euclidienne ($q = 2$)

$$d_2(\boldsymbol{x}, \boldsymbol{z}) = \sqrt{\sum_j (x^{(j)} - z^{(j)})^2}$$

Manhattan ($q = 1$)

$$d_1(\boldsymbol{x}, \boldsymbol{z}) = \sum_j |x^{(j)} - z^{(j)}|$$

Tchebychev ($q \to \infty$)

$$d_\infty(\boldsymbol{x}, \boldsymbol{z}) = \max_j |x^{(j)} - z^{(j)}|$$

Idée

Les voisins sont les observations auxquelles correspondent les k distances les plus faibles.

Exemple KNN : calculer les distances

Nouvelle observation $Q = (3, 2)$

On dispose des observations suivantes :

Observation	Coordonnées	Classe
A	$(1, 1)$	Non
B	$(2, 1)$	Oui
C	$(4, 3)$	Oui
D	$(5, 4)$	Non
E	$(3, 4)$	Oui

Avec la distance euclidienne, pour $Q = (3, 2)$:

$$d(Q, A) = \sqrt{(3-1)^2 + (2-1)^2} = \sqrt{5} \approx 2{,}236.$$

$$d(Q, B) = \sqrt{2}, \quad d(Q, C) = \sqrt{2}, \quad d(Q, E) = 2, \quad d(Q, D) = \sqrt{8}.$$

Exemple KNN : sélectionner les voisins

Trier les distances puis voter

Observation	Distance	Classe	Rang
B	$\sqrt{2} \approx 1{,}414$	Oui	1–2
C	$\sqrt{2} \approx 1{,}414$	Oui	1–2
E	2	Oui	3
A	$\sqrt{5} \approx 2{,}236$	Non	4
D	$\sqrt{8} \approx 2{,}828$	Non	5

Pour $k = 3$, les voisins sont B, C et E.

Prédiction

$$\widehat{y}(Q) = \mathrm{mode}(\mathsf{Oui}, \mathsf{Oui}, \mathsf{Oui}) = \boxed{\mathsf{Oui}}.$$

À discuter

Que pourrait-il se passer si l'on choisissait $k = 1$ ou $k = 5$?

Supposons que les trois voisins les plus proches aient les valeurs cibles :

$$y_{(1)} = 120, \qquad y_{(2)} = 130, \qquad y_{(3)} = 110.$$

Pour $k = 3$:

$$\widehat{y} = \frac{120 + 130 + 110}{3} = 120.$$

Une variante pondère les voisins par leur distance :

$$\widehat{y}(\boldsymbol{x}) = \frac{\sum_{i \in N_k(\boldsymbol{x})} w_i y_i}{\sum_{i \in N_k(\boldsymbol{x})} w_i}, \qquad w_i = \frac{1}{d(\boldsymbol{x}, \boldsymbol{x}_i) + \varepsilon}.$$

- Un voisin très proche reçoit un poids plus élevé.
- $\varepsilon > 0$ évite une division par zéro.
- Le choix de la pondération est lui aussi un choix de modèle.

Comment choisir le nombre de voisins k ?

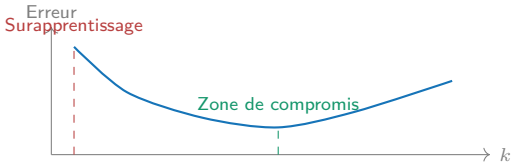
Compromis entre biais et variance

k petit

- Décision très locale.
- Forte sensibilité au bruit.
- Variance potentiellement élevée.
- Risque de surapprentissage.

k grand

- Décision plus lissée.
- Les détails locaux sont atténués.
- Biais potentiellement plus élevé.
- Risque de sous-apprentissage.



La valeur de k se sélectionne sur les données d'entraînement/validation, typiquement par validation croisée – jamais en optimisant directement sur le jeu de test.

Considérons deux caractéristiques : l'âge (années) et le revenu (dirhams).

$$\boldsymbol{x} = (30,\, 50\,000), \qquad \boldsymbol{z} = (31,\, 51\,000).$$

La distance euclidienne brute vaut :

$$d(\boldsymbol{x}, \boldsymbol{z}) = \sqrt{(30-31)^2 + (50\,000 - 51\,000)^2} \approx 1000.$$

La différence de revenu domine numériquement la différence d'âge.

Standardisation

Pour chaque caractéristique numérique j :

$$z^{(j)} = \frac{x^{(j)} - \mu_j}{\sigma_j}.$$

Les moyennes μ_j et écarts-types σ_j doivent être estimés uniquement sur les données d'entraînement (ou à l'intérieur de chaque fold de validation croisée).

- **Dimensionnalité :** dans un espace de grande dimension, les distances peuvent devenir moins discriminantes et les notions de voisinage perdent en efficacité.
- **Variables non pertinentes :** elles peuvent déformer les distances et donc modifier les voisins sélectionnés.
- **Mémoire :** le modèle conserve les observations d'entraînement.
- **Prédiction :** une recherche naïve compare une requête aux m observations ; son coût est de l'ordre de $O(mp)$ par requête pour p caractéristiques.
- **Données mixtes :** le traitement des variables catégorielles et leur encodage influencent la définition de la proximité.

À retenir

KNN est simple à décrire, mais la qualité de ses prédictions dépend fortement de la représentation des données et de la métrique utilisée.

Comparer les trois méthodes

Le mécanisme détermine les besoins et les compromis

	Decision Tree	Random Forest	KNN
Principe	Tests et partitions successives	Agrégation d'arbres randomisés	Proximité des observations
Apprentissage	Construction de règles	Bootstrap + arbres	Mémorisation du train
Prédiction	Parcours jusqu'à une feuille	Vote ou moyenne	Vote ou moyenne des voisins
Échelle des variables	Normalisation inutile	Normalisation inutile	Mise à l'échelle importante
Interprétabilité	Plutôt élevée	Plus limitée	Locale, peu explicite
Mémoire	Arbre appris	Plusieurs arbres	Données d'entraînement
Risque principal	Instabilité, surapprentissage	Coût, opacité relative	Échelle et dimension

Aucune méthode n'est universellement meilleure

Le choix dépend des données, de leur représentation, du coût de calcul et de la métrique adaptée au problème.

Problème

À partir des informations relatives à une campagne de marketing bancaire, prédire si un client souscrira un dépôt à terme (cible binaire).

Expériences à réaliser :

1. Entraîner un arbre et visualiser ses premières règles.
2. Faire varier la profondeur et observer train/validation.
3. Entraîner une forêt et étudier l'effet du nombre d'arbres.
4. Entraîner KNN avec différentes valeurs de k.
5. Comparer les trois modèles sur le même protocole.

Évaluation

- Matrice de confusion
- Précision
- Rappel
- F1-score
- ROC-AUC et PR-AUC

Précaution importante

Ne pas utiliser `duration` pour prédire avant l'appel : cette variable n'est connue qu'après celui-ci.

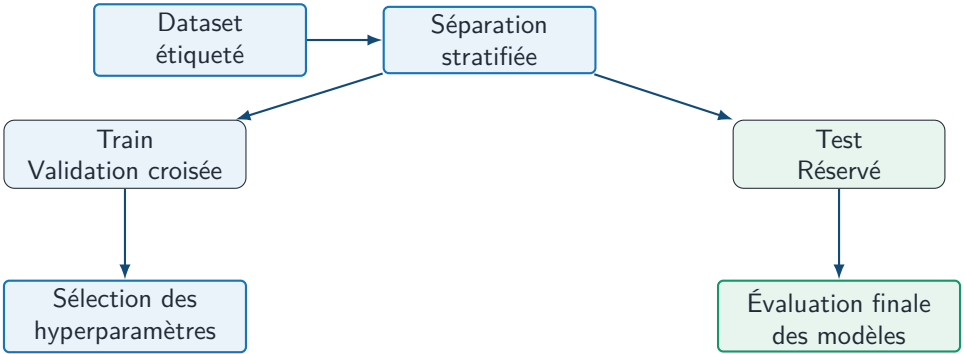
1. **Prise en main :** chargement, description des variables, cible et déséquilibre des classes.
2. **Préparation :** séparation stratifiée train/test ; prétraitements intégrés à des pipelines.
3. **Baseline :** DummyClassifier pour obtenir une référence minimale.
4. **Arbre :** entraînement, visualisation, profondeur et taille minimale des feuilles.
5. **Forêt :** entraînement, nombre d'arbres et importance des variables.
6. **KNN :** normalisation, variation de k et comparaison des métriques.
7. **Évaluation :** mêmes folds de validation, mêmes données de test, métriques adaptées au déséquilibre.
8. **Interprétation :** erreurs, temps d'entraînement, temps de prédiction et limites.

Règle expérimentale

Le jeu de test est réservé à l'évaluation finale. Toute transformation apprise (imputation, standardisation, encodage) doit être ajustée uniquement sur le sous-ensemble d'entraînement correspondant.

Protocole équitable de comparaison

Éviter les conclusions trompeuses



- Même partition train/test et mêmes folds pour les trois approches.
- Même cible et même ensemble d'informations disponibles au moment réel de la prédiction.
- Pipelines distincts : la normalisation est utile à KNN, pas nécessaire aux arbres.
- Ne pas retenir un modèle uniquement à partir de l'accuracy si les classes sont déséquilibrées.

Vérification des acquis

Questions à traiter après les démonstrations

1. Pour un arbre de régression, pourquoi la moyenne est-elle la valeur optimale d'une feuille lorsque le critère est la somme des erreurs quadratiques ?
2. Pourquoi CART teste-t-il les milieux entre valeurs distinctes consécutives ?
3. Quelle est la différence entre bootstrap et sélection aléatoire de variables dans une forêt ?
4. Pourquoi l'agrégation de plusieurs arbres corrélés peut-elle apporter un gain limité ?
5. En KNN, que change le passage de $k = 1$ à un k beaucoup plus grand ?
6. Pourquoi la standardisation doit-elle être effectuée à l'intérieur du pipeline de validation croisée ?

Question de synthèse

Pour une nouvelle tâche, quelles propriétés des données et quelles contraintes pratiques vous conduiraient à essayer d'abord un arbre, une forêt ou KNN ?

Références pour approfondir

Fondements des trois méthodes


L. Breiman, J. Friedman, R. Olshen, C. Stone,
Classification and Regression Trees, Wadsworth, 1984.


L. Breiman,
"Random Forests", *Machine Learning*, 45, 5–32, 2001.


T. Cover and P. Hart,
"Nearest neighbor pattern classification", *IEEE Transactions on Information Theory*, 13(1), 21–27, 1967.


T. Hastie, R. Tibshirani, J. Friedman,
The Elements of Statistical Learning, 2nd ed., Springer, 2009.


Scikit-learn documentation,
Decision Trees, Random Forests, and Nearest Neighbors.

À retenir

Un arbre **partitionne** l'espace des observations.

Une forêt **agrège** plusieurs arbres.

KNN **compare** une nouvelle observation à ses voisins.